

Ingénierie des Systèmes d'Information et Big Data

Module : Bases de Données Avancées

Projet de gestion de fournisseurs dans une PME

Présenté par :

EL MESSAOUDI Aya

Encadrant pédagogique

M. Hamid HRIMECH

Présenté le **01 juin 2026**

Année universitaire : 2025 – 2026

Table des matières

Remerciements	2
1 Contexte général	3
2 Objectifs fonctionnels et techniques	3
3 Schéma relationnel de la table fournisseur	4
4 Script SQL de déploiement	4
5 Rôle de l'ORM (Object-Relational Mapping)	4
6 Classe d'entité et fichier de mappage XML	5
7 Diagramme de classes UML	5
8 Architecture modulaire basée sur le composant TabPane	6
9 Descriptif détaillé des modules de l'application	6
9.1 1. Onglet Accueil et Authentification (Connexion)	6
9.2 2. Onglet Formulaire (Opérations d'Écriture CRUD)	7
9.3 3. Onglet Liste des Fournisseurs (Visualisation interactive)	7
9.4 4. Onglet Dashboard (Indicateurs décisionnels)	7
10 Rôle du contrôleur JavaFX	7
11 Gestion du filtrage en temps réel	8
12 Synchronisation et sélection dans la grille	8
13 Conclusion	9

Remerciements

Je tiens à exprimer mes sincères remerciements à notre professeur, Monsieur Hamid HRIMECH, qui nous a dispensé ce cours tout au long du semestre et qui est à l'origine de ce projet. Ses enseignements précieux, ses directives claires et ses exigences académiques nous ont permis de développer nos compétences et de mener à bien ce travail pratique. Qu'il trouve ici l'expression de notre profond respect et de notre gratitude pour son investissement pédagogique.

1 Contexte général

Dans le tissu économique actuel, la réactivité et la fiabilité des systèmes d'information constituent des leviers de performance majeurs pour les Petites et Moyennes Entreprises (PME). Le présent projet s'inscrit dans le cadre de la refonte du système de gestion d'une PME spécialisée dans la distribution alimentaire. Traditionnellement, le suivi des partenaires et des flux logistiques y était opéré via des fichiers de tableurs hétérogènes (Microsoft Excel). Cette approche montrait d'importantes limites : redondance des informations, risques élevés de saisies erronées, absence de concurrence d'accès et difficultés à produire des indicateurs statistiques fiables en temps réel.

Pour pallier ces faiblesses, la direction a initié le développement d'un prototype d'application lourde de bureau (Desktop). Conformément aux options de flexibilité proposées dans le cahier des charges pédagogique, notre choix s'est porté sur la **Gestion des Fournisseurs** plutôt que celle des produits. Les fournisseurs représentent en effet le premier maillon critique de la chaîne logistique (Supply Chain) ; une maîtrise rigoureuse de leurs coordonnées, de leur localisation géographique et de leur statut d'activité est fondamentale pour optimiser les processus d'approvisionnement.

2 Objectifs fonctionnels et techniques

L'application développée se doit de répondre à des exigences strictes en matière d'ergonomie et d'architecture logicielle :

- **Centralisation et Persistance** : Offrir un stockage sécurisé et structuré de l'ensemble des attributs des fournisseurs au sein d'un système de gestion de base de données relationnel (SGBDR).
- **Opérations CRUD Complètes** : Permettre aux utilisateurs finaux d'AJOUTER, MODIFIER, et SUPPRIMER des fiches fournisseurs de manière intuitive, sans nécessiter de connaissances en requêtage SQL.
- **Interface Riche et Segmentée** : Structurer l'application en modules distincts via un système d'onglets unifiés (Accueil, Formulaire de saisie, Liste globale interactive, Tableau de bord décisionnel).
- **Recherche et Filtrage Dynamiques** : Implémenter un mécanisme de recherche prédictive permettant de filtrer les fournisseurs en temps réel lors de la saisie d'un mot-clé (nom ou ville).
- **Architecture MVC** : Séparer strictement la structure de présentation (fichiers FXML), la logique de contrôle (classes Controller) et le modèle de données (classes d'entités persistantes).

3 Schéma relationnel de la table fournisseur

La base de données sous-jacente, nommée `gestion_fournisseurs`, a été implémentée sous le SGBDR MySQL. Elle s’articule principalement autour de la table `fournisseur`. La structure a été pensée pour allier performance d’indexation et flexibilité métier.

TABLE 1 – Dictionnaire des données de la table fournisseur

Nom du Champ	Type de Données	Contraintes	Description
<code>id</code>	INT	PRIMARY KEY, AUTO_INCREMENT	Clef primaire technique interne.
<code>code</code>	VARCHAR(50)	NOT NULL, UNIQUE	Identifiant unique métier (ex : F001).
<code>nom</code>	VARCHAR(100)	NOT NULL	Raison sociale ou nom complet.
<code>telephone</code>	VARCHAR(20)	-	Ligne téléphonique directe.
<code>email</code>	VARCHAR(100)	-	Adresse de messagerie électronique.
<code>ville</code>	VARCHAR(50)	-	Ville d’implantation principale.
<code>actif</code>	BOOLEAN	DEFAULT TRUE	Statut opérationnel (archivage logique).
<code>categorie_id</code>	INT	-	Clef de classification métier.

4 Script SQL de déploiement

Afin de concrétiser le modèle relationnel conçu, un script de déploiement SQL a été développé et exécuté sous le SGBD MySQL. Ce script prend en charge la création automatisée de la base de données nommée `gestion_fournisseurs` avec un encodage universel UTF-8 pour la gestion des caractères. Il assure également la génération de la table fournisseur en définissant rigoureusement les types de données (VARCHAR, INT, BOOLEAN), les contraintes d’intégrité (clés primaires en auto-incrément, unicité du code fournisseur) ainsi que les valeurs par défaut pour l’archivage logique des données.

5 Rôle de l’ORM (Object-Relational Mapping)

Afin de s’affranchir de l’écriture fastidieuse de requêtes SQL imbriquées dans le code Java (via JDBC classique), nous avons intégré le framework ORM **Hibernate 4.3.x**. Celui-ci assure la translation automatique entre les lignes de la table relationnelle de la base de données et les instances d’objets en mémoire vive. L’utilisation d’Hibernate garantit également une portabilité accrue de l’application vis-vis d’autres systèmes de bases de données (Oracle, PostgreSQL) par simple modification du dialecte.

6 Classe d'entité et fichier de mappage XML

Conformément à la structure du projet, la correspondance est gérée par le fichier `Fournisseurs.hbm.xml`. Il associe la classe Java Plain Old Java Object (POJO) `Fournisseurs.java` (contenant les attributs, constructeurs, getters et setters) aux colonnes de la base de données. Les sessions de transactions (ouverture, commit, rollback en cas d'anomalie) sont centralisées au sein d'une classe utilitaire nommée `NewHibernateUtil.java`.

7 Diagramme de classes UML

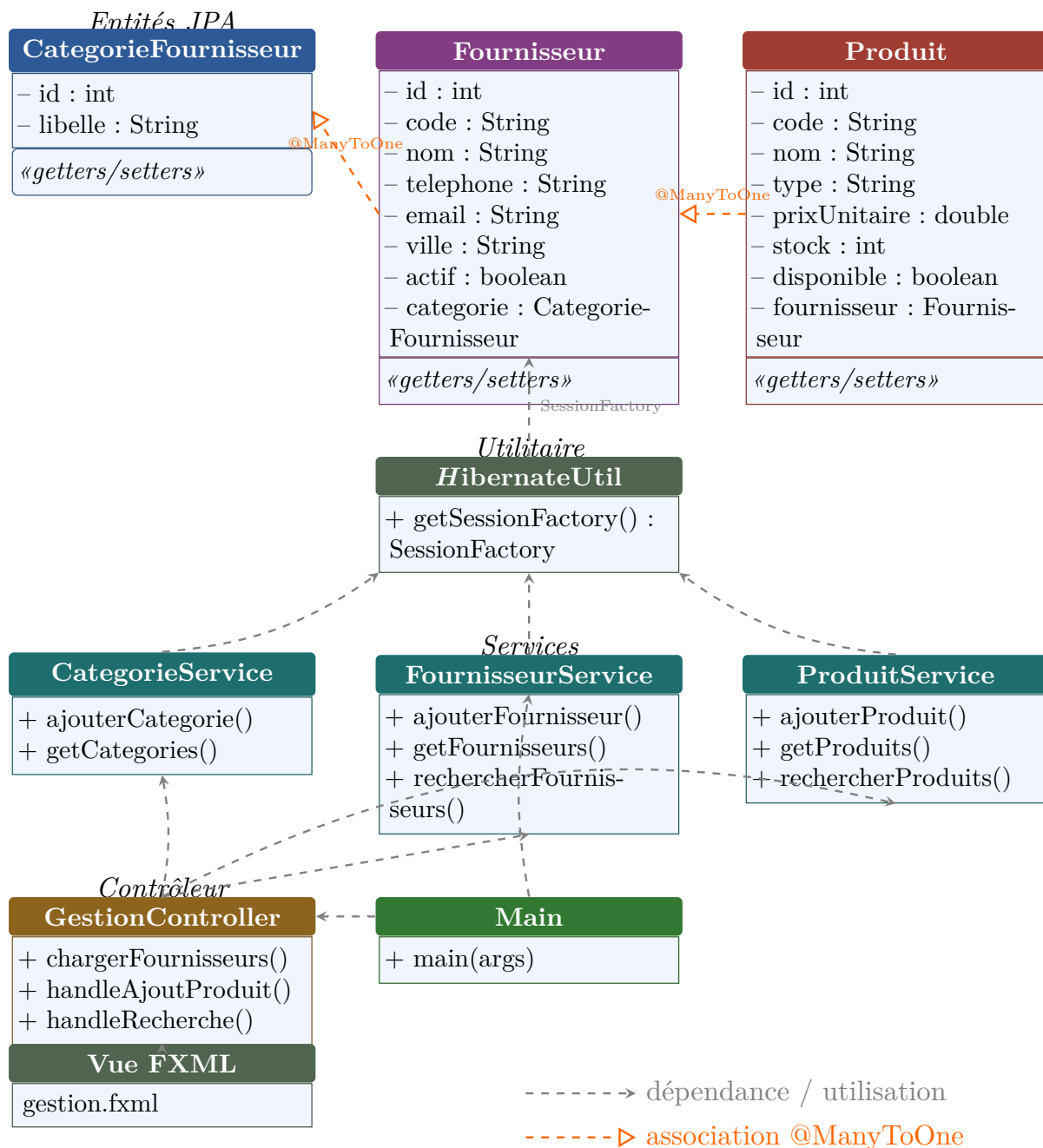


FIGURE 1 – Diagramme de classes – Gestion des fournisseurs

8 Architecture modulaire basée sur le composant `TabPane`

L'interface utilisateur a été développée en langage déclaratif **FXML** via l'outil `Scene Builder` et intégrée nativement dans l'environnement `NetBeans IDE 8.2`. Pour assurer une navigation fluide et éviter la multiplication fastidieuse de fenêtres `pop-up`, le conteneur racine de l'application est un `TabPane`. Chaque onglet encapsule une vue fonctionnelle métier spécifique.

9 Descriptif détaillé des modules de l'application

9.1 1. Onglet Accueil et Authentification (Connexion)

Cet écran sert de barrière de sécurité initiale pour l'accès aux données. Divisé verticalement en deux sections harmonieuses, il arbore à gauche un panneau bleu arborant le logo de l'entreprise et l'intitulé de l'application (« Gestion Fournisseurs »). La section droite intègre un formulaire de connexion épuré avec des champs de saisie pour l'identifiant (`Username`) et le mot de passe (`Password`), surmontés d'une icône d'utilisateurs et d'un bouton d'action bleu clair pour déclencher l'authentification.

9.2 2. Onglet Formulaire (Opérations d'Écriture CRUD)

Cet onglet constitue l'espace de saisie et de manipulation des fiches individuelles. Il se compose de six zones de texte disposées en grille pour une lecture optimale (*Code Fournisseur*, *Nom du Fournisseur*, *Téléphone*, *Email*, *Ville*, et *Catégorie ID*). Un panneau latéral droit affiche un « Guide rapide » textuel rappelant le mode opératoire de l'application pour guider l'utilisateur.

La validation des opérations s'effectue via quatre boutons d'action dotés d'un code couleur explicite conforme aux standards ergonomiques :

- **+ Ajouter (Bleu)** : Insère un nouveau fournisseur en base après validation.
- **Modifier (Vert)** : Répercute les modifications textuelles sur l'enregistrement sélectionné.
- **Supprimer (Rouge)** : Retire l'élément de la base (ou désactive son statut).
- **Annuler (Gris foncé)** : Réinitialise l'ensemble des champs du formulaire.

9.3 3. Onglet Liste des Fournisseurs (Visualisation interactive)

Il intègre un grand composant `TableView` interconnecté à une `ObservableList` JavaFX. Chaque colonne correspond point par point à un attribut du modèle de données. En haut de cet écran, une barre de recherche textuelle associée à un bouton « Actualiser » permet de filtrer instantanément la liste des fournisseurs affichés en saisissant simplement les premières lettres d'un nom ou d'une ville.

9.4 4. Onglet Dashboard (Indicateurs décisionnels)

Ce volet apporte une forte valeur ajoutée analytique pour les gestionnaires de la PME. Il affiche de manière synthétique deux tuiles de métriques clés : le *Nombre Total de Fournisseurs* (fixé à 14 dans notre jeu de données actuel) et le nombre de *Villes distinctes* (13 villes). En dessous de ces indicateurs, un composant graphique circulaire `PieChart` affiche dynamiquement la répartition géographique des fournisseurs, illustrant visuellement l'ancrage territorial des partenaires commerciaux de la PME.

10 Rôle du contrôleur JavaFX

La classe Java `fournisseursController.java` centralise la logique métier et fait office de passerelle exclusive entre les interactions graphiques de l'utilisateur sur la vue FXML et la couche de persistance Hibernate. Les éléments graphiques y sont injectés via l'annotation standard `@FXML`.

11 Gestion du filtrage en temps réel

Pour répondre à l'exigence d'optimisation de l'expérience utilisateur, un filtre prédictif a été mis en œuvre. Plutôt que de lancer une requête lourde vers la base de données MySQL à chaque caractère saisi, nous utilisons un objet filtré encapsulé côté client : `FilteredList`.

Le principe algorithmique repose sur l'écoute active (Listener) des changements de propriétés textuelles du champ de recherche. Dès qu'une modification survit, le prédicat de la liste est réévalué. L'expression de correspondance peut être formalisée mathématiquement comme suit : Soit $\mathcal{F}$ l'ensemble des fournisseurs, et k le mot-clé saisi par l'utilisateur. Un fournisseur $f \in \mathcal{F}$ est conservé dans l'affichage si et seulement si :

$$P(f) = (f.nom \times k) \vee (f.ville \times k) \quad (1)$$

où l'opérateur $\times$ désigne ici l'inclusion textuelle insensible à la casse (sous-chaîne de caractères).

12 Synchronisation et sélection dans la grille

Un autre point fort de l'implémentation réside dans la synchronisation bidirectionnelle lors de la sélection d'une ligne au sein du tableau de l'onglet Liste. Le contrôleur intercepte l'index sélectionné via un écouteur sur le modèle de sélection :

```

1 tableView.getSelectionModel().selectedItemProperty().
  addListener((obs, oldSelection, newSelection) -> {
2     if (newSelection != null) {
3         txtCode.setText(newSelection.getCode());
4         txtNom.setText(newSelection.getNom());
5         txtTelephone.setText(newSelection.getTelephone());
6         txtEmail.setText(newSelection.getEmail());
7         txtVille.setText(newSelection.getVille());
8         txtCategorie.setText(String.valueOf(newSelection.
          getCategorieId()));
9     }
10 });

```

1 – Code d'écoute de la sélection sur le TableView

Cette approche permet à l'opérateur de sélectionner un fournisseur dans la liste globale, puis de basculer instantanément sur l'onglet Formulaire pour procéder à une mise à jour immédiate.

13 Conclusion

Le prototype fonctionnel d'application de bureau pour la **Gestion des Fournisseurs** mené à bien au cours de ce projet démontre l'efficacité de l'alliance des technologies JavaFX et Hibernate pour répondre aux problématiques concrètes de modernisation des SI des PME.

L'utilisation d'un framework d'ORM a considérablement accéléré le cycle de développement en éliminant la plomberie SQL manuelle et en garantissant une étanchéité parfaite de la couche de persistance. Côté présentation, l'exploitation fine des composants de JavaFX (*TabPane*, *TableView* couplé aux *FilteredLists*, et graphiques de type *PieChart*) a permis de doter l'entreprise d'un outil moderne, hautement interactif et prêt pour l'aide à la décision.

En guise de perspectives d'évolution, cette application pourrait facilement être enrichie par l'adjonction d'un module d'exportation automatisé des listes de partenaires au format PDF ou encore par la mise en place d'un système de gestion fine des rôles et des habilitations des utilisateurs.